Homework 1 Report

Games for Graph Problems

學號：314581030

---

1. 作業目標與整體說明

本次作業的目標，是嘗試用 **game-theoretic approach** 的方式來求解圖論問題。根據作業說明，玩家會根據其他玩家目前的策略，不斷調整自己的行動，選擇對自己最有利的 **best response**，直到整體系統到達一個沒有人願意再單方面改變策略的穩定狀態，也就是 **Nash Equilibrium (NE)**。

本作業共分成三個部分。依照我的學號 **314581030** 計算後：

- `314581030 mod 5 = 0`，因此 **Problem Set 1** 需要實作 **MIS-based IDS Game**。
- `314581030 mod 2 = 0`，因此 **Problem Set 2** 需要實作 **Symmetric MDS-based IDS Game**。
- **Problem Set 3** 則是每位同學都要完成的 **Matching Game / Maximal Matching**。

除了完成三題 solver 之外，作業也要求自行生成 **Watts-Strogatz (WS) model** 圖，並利用這些圖來測試方法的表現與穩定性，最後整理成報告。

- 註1: 本篇Report透過Prompt請ChatGPT先為我將對話內容整理成Report，並且後續再以人工去進行修正，一樣有將產生Report 所用之Prompt附上。'
- 註2: 我主要是以和chatGPT討論出架構和概念，使其產生程式碼架構的Prompt，然後讓Codex去生成完整程式碼，確保其程式碼的可靠度。

---

2. Code Description

2.1 程式架構

本次作業我將程式拆成三個主要部分：

- `314581030_HW1_main.py` 作為主程式，負責讀入圖的輸入格式、建立 graph 結構，並依序執行三個 graph game solver，最後輸出三題答案。

- `ws_generator.py` 負責生成 WS model graph，作為額外測試資料來源。

- `checker.py` 作為本機驗證與 benchmark 工具，用來檢查 solver 輸出的結果是否合法，並觀察不同參數下的 cardinality、move_count 與正確率等資訊。這支程式主要用於分析與除錯，不屬於正式提交主程式的必要部分。

2.2 輸入與輸出格式

根據作業說明，助教會透過 command-line arguments 將圖輸入程式。輸入格式為：第一個數字代表節點數 `n`，接著是 `n` 個長度為 `n` 的 bit string，表示 adjacency matrix。

例如 given test case 為：

```
6 010000 101100 010010 010010 001101 000010
```

主程式會讀入此格式後建立圖，並依序執行三個 solver，最後輸出三題 cardinality。

2.3 Graph 資料結構

在實作上，我先將輸入的 bit strings 轉成：

- adjacency matrix
- adjacency list

再封裝成 `Graph` 類別，提供以下基本操作：

- `degree(i)`：回傳節點 `i` 的 degree
- `neighbors(i)`：回傳節點 `i` 的鄰居
- `is_edge(i, j)`：判斷 `i` 與 `j` 是否有邊相連

這樣做的原因是三題都會反覆用到：

- 鄰居查詢
- degree 比較
- edge的存在性檢查

因此統一成一個 graph abstraction 之後，後面的 solver 與 checker 都會比較容易撰寫與維護。

2.4 Problem 1：MIS-based IDS Game

(1) 狀態表示

本題使用 bit vector 表示整體狀態：

$C=(c_1,c_2,\dots,c_n),\quad c_i\in{0,1}$

其中：

- $c_i = 1$ 代表節點 `i` 被選進集合
- $c_i = 0$ 代表節點 `i` 沒被選進集合

---

(2) Utility 與 best response

老師提供上課講義的定義，對每個節點 `i`：

$L_i={j\mid j\in N_i,\ \deg(j)\ge \deg(i)}$

也就是說，L_i 是 `i` 的鄰居中，degree 大於等於 `i` 的那些節點。 他的 utility function 為：

$u_i(C)=c_i\left(1-\alpha \sum_{j\in L_i} c_j\right),\quad \alpha>1$

這個 utility 的直觀意義是：若一個節點被選進集合，但附近已經有 degree 不小於自己的節點也被選中，則它會因為重複選擇而受到懲罰。換句話說，這個設計鼓勵在衝突時優先保留高 degree 節點，讓較低 degree 的節點退出。

對應的 best response 可以簡化為：

- 若 L_i 中已存在被選中的節點，則 `i` 的最佳回應是退出（設為 0）
- 否則 `i` 的最佳回應是進入集合（設為 1）

---

(3) Dynamics 與 multi-start

本題採用 asynchronous best-response dynamics。實作上，我會先給定一個 initial state，接著找出所有目前狀態不等於 best response 的節點，再從中隨機選出一個玩家更新狀態。此過程持續進行，直到無人想改變策略，或者超過步數上限。

由於不同 initial state 與 tie-breaking 可能收斂到不同結果，因此我採用了 multi-start 的做法，包括：

- all-zero state
- all-one state
- 多個 random 初始 bit vector

最後從所有合法結果中，選擇 **minimum cardinality** 的解作為輸出，因為 IDS / dominating set 類問題的目標是盡量縮小所選集合的大小。

---

(4) 合法性檢查

本題最終結果至少需滿足 **Independent Dominating Set (IDS)** 的條件：

- **Independent**：任兩個相鄰節點不能同時被選
- **Dominating**：每個節點不是自己被選，就是至少有一個鄰居被選
- 藉由重新用另一隻程式去確認IDS性，然後用不同agent產生(避免幻覺)，去做verification，確保程式碼上的正確性。

2.5 Problem 2：Symmetric MDS-based IDS Game

(1) 狀態表示

與 Problem 1 相同，本題也使用 bit vector：

$C=(c_1,c_2,\dots,c_n),\quad c_i\in{0,1}$

表示每個節點是否被選入集合。

---

(2) Utility 設計概念

與 Problem 1 相比，Problem 2 並不是透過 degree 優先權來決定留下誰，而是直接在 utility 中同時考慮：

1. **有效支配的獎勵**
2. **被選進集合的成本**
3. **違反 independence 的懲罰**

對每個節點 `i`，定義：

$M_i=N_i\cup{i}$

也就是自己與所有鄰居的 closed neighborhood。接著定義：

$v_i(C)=\sum_{j\in M_i} c_j$

表示節點 `i` 目前被多少個 selected nodes 支配。

然後再定義 gain：

$g_i(C)=\begin{cases} \alpha, & \text{if } v_i(C)=1\ 0, & \text{otherwise} \end{cases}$

其意思是：只有當一個點剛好被一個 selected node 支配時，才獲得獎勵。 若沒有人支配，或被多個節點重複支配，都不算理想情況。

此外，為了維持 independent set 的性質，還定義 independence penalty：

$w_i(C)=\sum_{j\in N_i} c_i c_j \gamma$

當一個節點被選進集合，且其鄰居也同時被選中時，就會受到較大的懲罰。

最後，本題 utility 寫為：

$u_i(C)= \begin{cases} \sum_{j\in M_i} g_j(C)-\beta-w_i(C), & \text{if } c_i=1\ 0, & \text{if } c_i=0 \end{cases}$

這個設計的直觀想法是： 若我進入集合，我應該能對自己和鄰居帶來有效的支配貢獻；但同時，我進入集合本身要付出固定成本，而且若我與鄰居同時被選，還會因違反 independence 而被重罰。

---

(3) Best response 與 dynamics

本題不像 Problem 1 一樣有簡化後的 closed-form BR，因此實作上，我直接比較：

- `c_i = 0` 時的 utility
- `c_i = 1` 時的 utility

若 `utility(1) > utility(0)`，則 best response 為 1；若 `utility(0) > utility(1)`，則 best response 為 0；若相等，則以 random tie-breaking 處理。

本題同樣使用 asynchronous best-response dynamics 與 multi-start，並從多次 execution 中選擇 **minimum cardinality** 的合法解作為輸出。

---

(4) 合法性檢查

本題最終仍需滿足 IDS 的條件，因此 checker 與 Problem 1 類似，也會檢查：

- 是否 independent
- 是否 dominating
- 是否為合法 IDS

2.6 Problem 3：Matching Game / Maximal Matching

(1) 問題本質

本題與前兩題最大的不同，在於它不是集合選點問題，而是 **matching** 問題。 每個節點是一位玩家，每位玩家可以：

- 選擇某個鄰居作為提案對象
- 或選擇 `null`

只有當兩個相鄰節點互相選擇對方時，才形成真正的 matched pair。

作業要求的是 **maximal matching**，也就是說：最後的 matching 不一定要是全域最大的，但至少不能再加入任何額外的 matching edge。

- 我的直覺想法:

    1. 先看有沒有人要跟自己提案，有的話就在這裡面挑鄰居最少的接受。
    2. 取當前鄰居最少的node，並且看有沒有任何他的鄰居是unmatched，然後在這之中一樣挑鄰居最少的那個作為matched的人，向它提案說要跟它matched。
    3. 重複上面的iteration應該就會從鄰居最少的一路到NE。

- 與AI討論後: 得到下面的Utility 設計。

---

(2) 狀態表示

本題中，對每個玩家 `i`：

- `state[i] = None`：表示不提案
- `state[i] = j`：表示向鄰居 `j` 提案

若：

- `state[i] = j`
- 且 `state[j] = i`

則 `(i,j)` 形成一條 matching edge。

---

(3) Utility 設計

由於本題需要自行設計 utility function，我採用的想法是： 讓成功的 mutual match 最有價值，向目前 unmatched 的鄰居提案次之，而明顯無法形成配對的單方面提案則給予較低分數。

具體來說：

- 若 `state[i] = None`，則 utility = 0

- 若 `state[i] = j` 且 `state[j] = i`，表示成功 mutual match，給高分
- 若 `state[i] = j` 且 `state[j] = None`，表示對目前 unmatched 的鄰居提案，給中等分數
- 其他情況（例如對方已指向別人、提案無望）則給負分

另外，根據教材中的 optional heuristic，我也加入了一點對 **degree 較小鄰居** 的偏好。因為度數較小的節點選擇通常較少，若先讓它們被匹配，可能有助於增加 matching 的大小。

---

(4) Best response 與 update heuristic

對於每位玩家，我會枚舉所有可能策略：

- `None`
- 所有鄰居

接著固定其他玩家的狀態，逐一計算 utility，選取 utility 最大的策略作為 best response。若有 tie，則優先選 degree 較小的鄰居，若仍相同則 random。

此外，在 dynamics 中更新玩家的順序上，我也加入了 heuristic：

1. 優先更新目前尚未成功 matched 的玩家
2. 在其中優先考慮 degree 較小者
3. 若仍有多個候選，再以 random 選擇

這樣做的目的，是希望模擬「先照顧比較難匹配的點」，使結果更容易形成品質較好的 maximal matching。

---

(5) Cardinality 與合法性

本題的 cardinality 定義為 **matched pairs 的數量**。例如若結果為兩條 matching edges，則 cardinality = 2。

在 checker 中，我將 final state 轉換成 matching edge set，並進一步檢查：

- strategy profile 是否 well-formed
- matching 是否合法
- 是否為 maximal matching

此外，我也特別觀察：

- 目前有多少 unmatched vertices
- 這些 unmatched vertices 的鄰居中是否還有 unmatched 的點

因為若存在這種情況，通常就表示目前 matching 仍可再加入新邊，因此並非 maximal matching。

2.7 WS Generator 與 Checker

除了 main solver 外，我另外實作了：

`ws_generator.py`

用於生成 **Watts-Strogatz model** graph。生成流程為：

1. 先建立 regular ring lattice
2. 每個節點連到左右最近的 `k/2` 個鄰居
3. 以機率 `p` 進行 rewiring
4. 保證圖中沒有 isolated node。

`checker.py`

作為本機驗證與 benchmark 工具，其主要功能包含：

- 呼叫 `ws_generator.py` 產生測資
- 對三題 solver 進行 exact checking
- 蒐集 benchmark metrics
- 繪製 cardinality、move_count、合法率等圖表

其中特別重要的檢查包括：

- Problem 1 / 2 是否為合法 IDS
- Problem 3 是否為合法 matching、是否為 maximal、是否到達 NE
- 是否存在 unmatched vertex 仍有 unmatched neighbor 的錯誤情況

2.8 Checker到底檢查了甚麼? 到底如何確保安全性?

錯誤類型	**checker 會不會容易抓到**
輸出不是 IDS（有邊兩端都是 1，或有節點沒被支配）	**會**，`valid_p1` / `valid_p2` 會掉。
輸出是 IDS 但 **不是** NE（有人可單邊改善）	**會**，`ne_p1` / `ne_p2` 會掉。
matching 不合法、或非極大、或非 NE	**會**，對應 `valid_p3`、`max_p3`、`ne_p3`。
演算法 **多算/少算步數**、或解**不是最優** 但仍是合法 IDS + NE	**不一定**；checker **沒有**普遍驗「最優」，主要看 **結構 + NE**。
bug 剛好讓輸出仍是 IDS 且仍是 NE（機率上可能，小圖較難）	**可能漏網**。

3. All Test Cases Used

3.1 Given test case

作業提供的 given test case 為：

```
6 010000 101100 010010 010010 001101 000010
```

根據投影片範例，對我的三題對應結果應為：

- MIS-based IDS Game：2

- Symmetric MDS-based IDS Game：2
- Matching Game：2

此測資作為最基本的 smoke test，用來確認主程式輸入輸出與三題 solver 是否能正常工作。

3.2 小型手動測資

除了 given test case 之外，我也使用一些小型圖作為手動驗證測資，例如：

(1) Path graphs

- P2
- P3
- P4
- P5

用途：

- 容易人工推導 IDS 與 matching 結果
- 適合檢查第三題的 maximal matching 是否正確

---

(2) Cycle graphs

- C4
- C5
- C6

用途：

- 測試對稱圖下的收斂行為
- 觀察不同 initial state 與 tie-breaking 是否導致不同終態

---

(3) Star graph

用途：

- 檢查 Problem 1 中 degree-based priority 的效果
- 也可觀察第三題中 degree bias 是否真的比較傾向優先照顧葉節點

---

(4) Complete / dense graphs

例如：

- K3
- K4

用途：

- 測試高衝突情況下的 best response、tie-breaking 與 matching legality

3.3 WS model 測資

為了進一步測試方法在不同圖結構下的表現，我使用 `ws_generator.py` 產生多組 WS graphs。主要測試方式包括：

固定 `k, p`，改變 `n`

例如：

- `n = 20, 30, 40`
- `k = 4`
- `p = 0.3`
- `trials = 10`（每個 x-value）

用途：

- 觀察圖規模變大時，cardinality 與 move_count 的變化

固定 `n, k`，改變 `p`

例如：

- `n = 40`
- `k = 4`
- `p = 0.0, 0.1, 0.2, 0.3, 0.5, 0.7, 1.0`
- `trials = 10`（每個 x-value）

用途：

- 觀察 rewiring probability 改變時，對結果與收斂性的影響

多個 random seeds

每組參數會搭配多個 seed 與 multiple trials， 以避免只觀察到單一隨機結果。

3.4 新增的 benchmark 腳本與輸出（本次更新）

本次我將測試流程集中在 `run_small_cases_checker.py`（不另外拆新檔），統一呼叫 `checker.py`，並新增以下能力：

1. **小型手動測資（`--trials 30`）**
    - 分類顯示 `Path / Cycle / Star / Dense-Complete / Given`
    - 每一類型同時顯示：
        - cardinality（3 題）
        - move_count（3 題）
        - Problem 3 額外驗證指標（`valid_p3`, `max_p3`, `ne_p3`, `unmatched/n`, `unmatch_nb/n`）
2. **WS benchmark（`--trials 10`）**

    - `n` sweep：`n = [20, 30, 40]`, `k=4`, `p=0.3`
    - `p` sweep：`p = [0.0, 0.1, 0.2, 0.3, 0.5, 0.7, 1.0]`, `n=40`, `k=4`
    - 同樣輸出 Problem 3 額外驗證指標，確認是否出現「unmatched 節點仍有 unmatched neighbor」的情況

對應輸出圖檔（位於 `checker_outputs/`）：

- `small_cases_by_type_trials30.png`
- `ws_vary_n_trials10.png`
- `ws_vary_p_trials10.png`

---

4. Result Analysis

4.1 為何選擇這些 metrics？

作業說明中提到，除了 selected set 的大小之外，也要留意 players' movements，也就是 move_count。因此本次分析主要觀察以下幾個指標。

4.2 Average cardinality

Problem 1 / Problem 2

這兩題都屬於 IDS / dominating set 類型，因此 cardinality 越小，代表使用越少節點就能完成支配，通常表示結果越理想。

因此 average cardinality 可用來檢查：

- utility function 是否能有效排除冗餘節點
- multi-start 是否有助於找到更小的合法解

Problem 3

第三題只要求 maximal matching，而非 maximum matching，但在大多數情況下，較大的 maximal matching 仍代表較好的配對品質。

因此 average cardinality 可用來觀察：

- 所設計的 utility 與 degree bias 是否傾向找到較大的 matching

4.3 Average move_count

這個指標反映的是 best-response dynamics 的收斂成本。 即使最後得到合法解，若過程中需要大量 player movements，也可能表示這套遊戲設計收斂較慢，或容易產生來回震盪。

因此 average move_count 可用來觀察：

- graph 規模變大時，收斂難度是否上升

- rewiring probability 改變後，是否影響收斂速度
- 不同 utility 設計是否穩定

- 註:然後我因為其中init state會不一樣，譬如全0，全1，然後tie的時候又會random先後動，所以需要多幾次trials比較不會被這些影響，我取30次。

4.4 Valid rate

valid rate 代表多次 execution 中，有多少比例真的得到合法解。

- Problem 1 / 2：合法 IDS 的比例
- Problem 3：合法 matching 的比例

這個指標的重要性在於： 若 average cardinality 看起來不錯，但 valid rate 很低，代表方法只是偶爾成功，穩定性不足。

4.5 Maximal rate（Problem 3）

第三題要求的是 maximal matching，因此除了 matching 合法之外，還必須確認結果是否 maximal。

maximal rate 用來觀察：

- 多次 execution 中，有多少比例真的達到 maximal matching
- 若 maximal rate 偏低，則表示 utility 設計或 dynamics 還有問題

4.6 Unmatched vertices 與 unmatched-neighbor 狀況

對第三題而言，單純看 matching cardinality 還不夠，還需要看：

- 最後有多少 unmatched vertices
- 其中有多少 unmatched vertices 的鄰居也 unmatched

第二個指標尤其重要。 如果存在 unmatched vertex 的鄰居也 unmatched，則代表仍有可能再加上一條 matching edge，因此目前解不應視為 maximal。這個資訊比單純的 True / False 更有診斷價值，因為它能指出錯誤的型態。

4.7 是否到達 NE

由於本作業強調的是 graph game 與 best-response dynamics，因此除了結果合法之外，也應檢查該結果是否為 utility 下的 Nash equilibrium。

- 若一個結果合法，但並不是 NE，表示它只是剛好對，但不一定是 game dynamics 真正穩定後的狀態
- 若一個結果同時合法且為 NE，才更符合題目原本想表達的 graph game 精神

4.8 預期觀察方向

Problem 1 與 Problem 2

- Problem 1 透過 degree priority 偏向保留高 degree 節點
- Problem 2 則直接透過 utility 中的 gain / cost / penalty 控制 local structure

可以比較兩題在：

- cardinality
- move_count
- valid rate

上的差異。

Problem 3

第三題主要觀察：

- utility 設計是否能穩定輸出合法 matching
- maximal rate 是否足夠高
- unmatched vertices 的情況是否合理
- 是否存在 unmatched vertex 仍有 unmatched neighbor

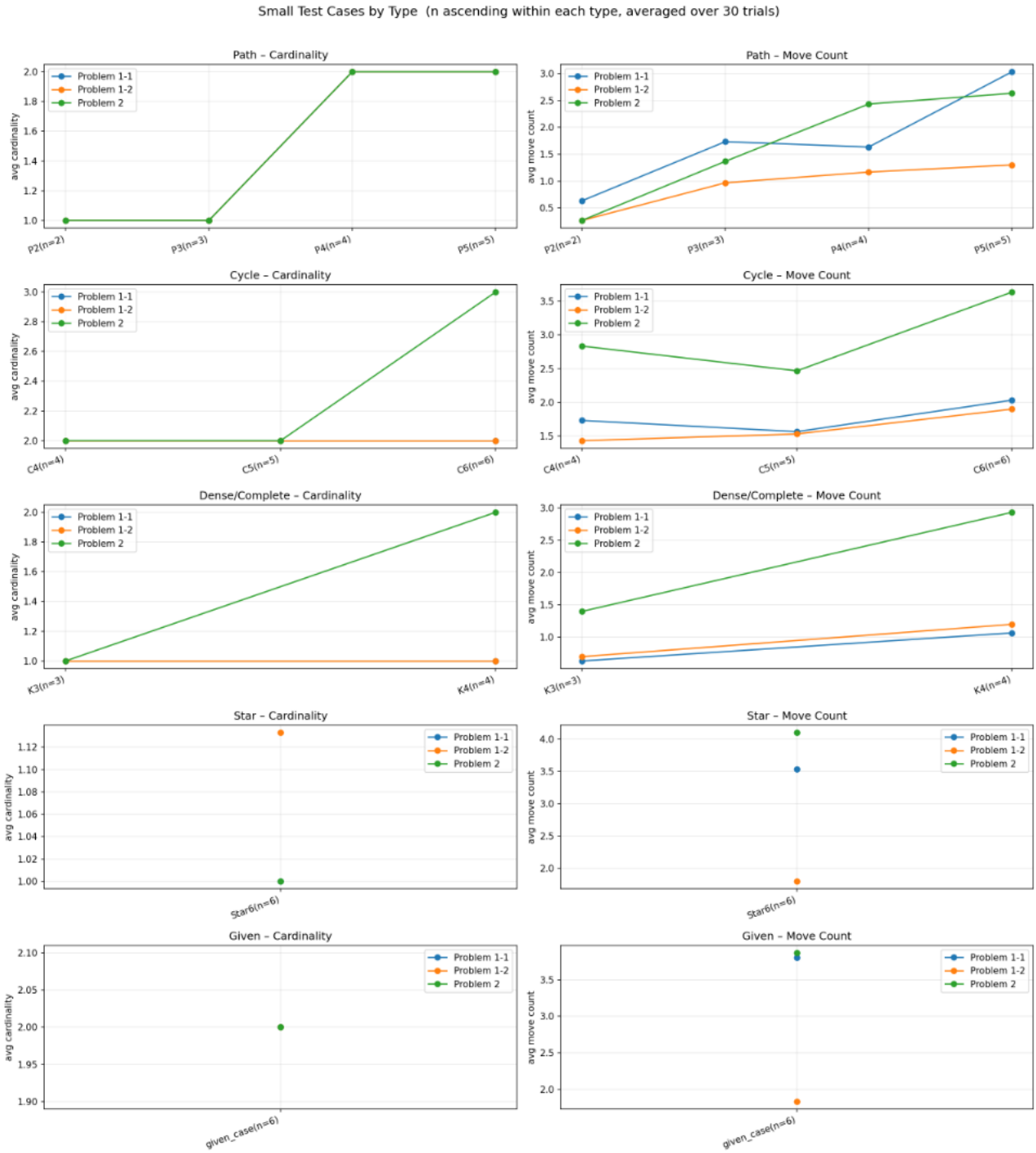
若最後一項經常為 0，則可視為本題 utility 與 update heuristic 設計是合理的。

WS parameter 的影響

- 當 n 增加時，預期 move_count 可能增加
- 當 p 增加時，圖結構變得更隨機，可能影響 local degree 分布與收斂行為
- 這些都可透過 benchmark 圖表進一步觀察

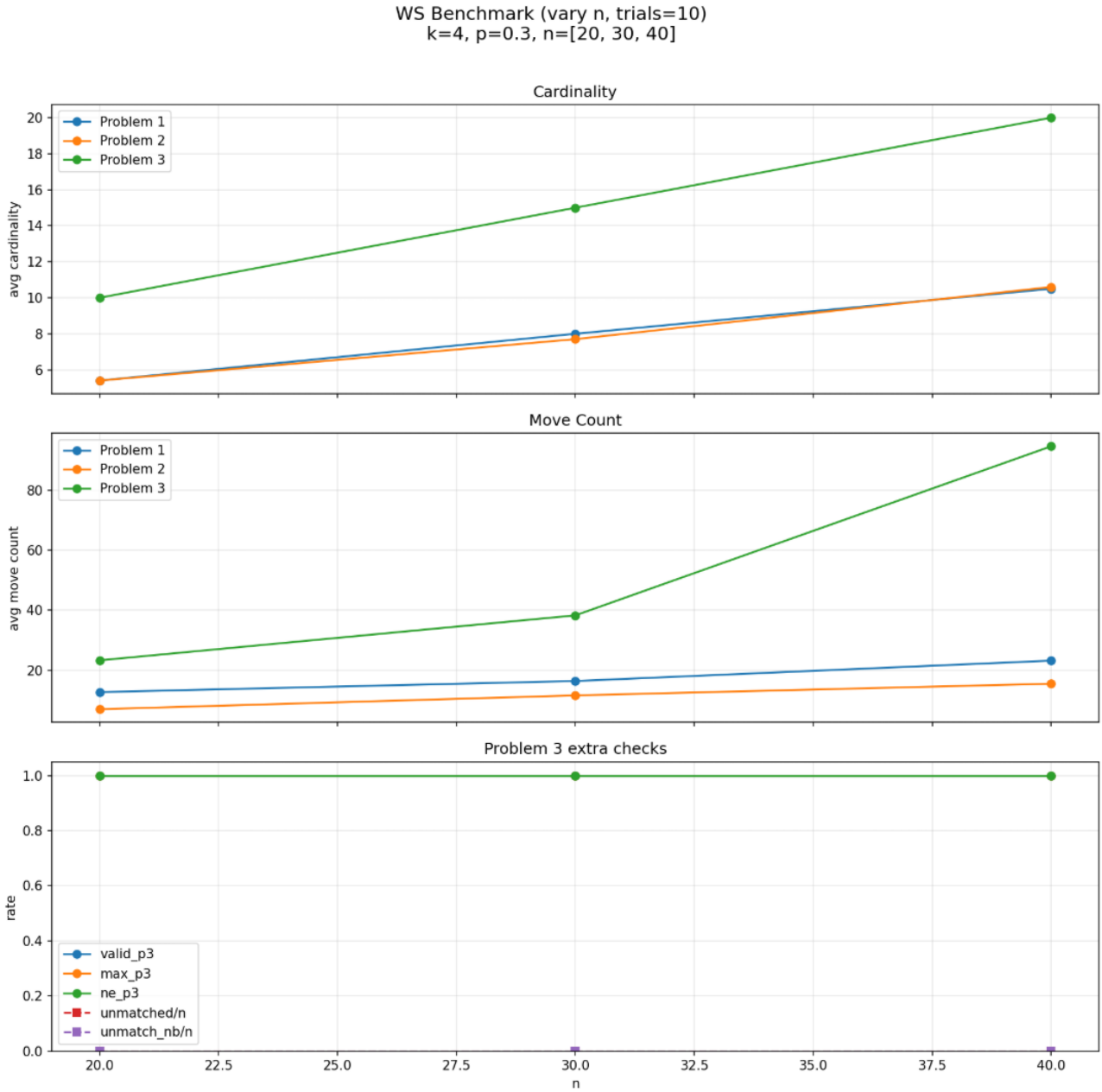
分析

1. Average cardinality/Average move_count



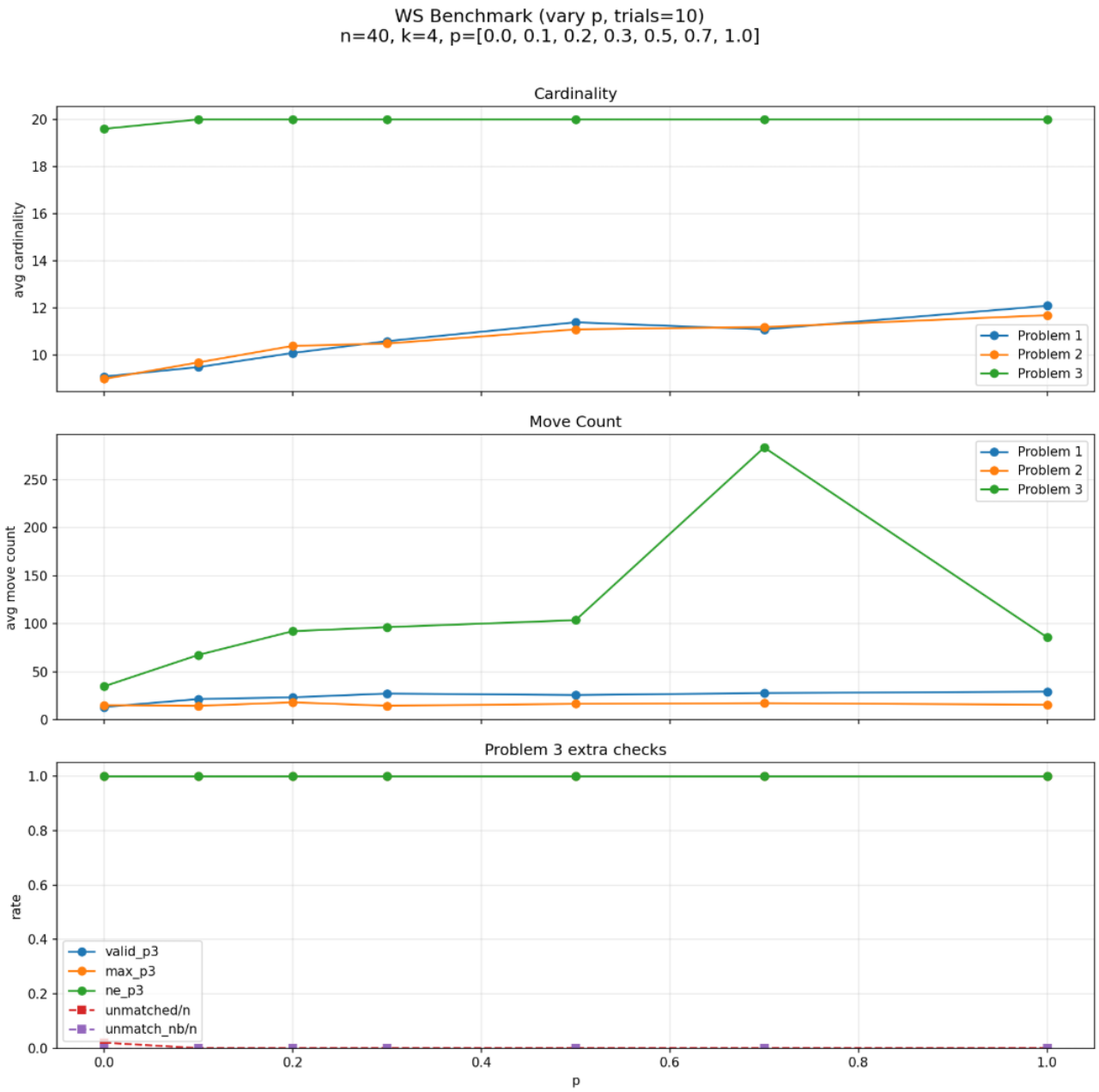
- 從各種的左圖來看，尤其Path，稍微手算一下就可以verify說確實他理論上到達NE應該要長這樣沒錯。
- 右圖來看確實隨著N的增加，到達NE的過程每個問題都會稍微增加難度，多走幾步。

2. WS different n



n 不同的結果其實跟前面的small test case有異曲同工，整體來說看起來就算是合理，而尤其matching問題最擔心的unmatch至少從測試結果圖來看表現都還算是不錯。 (註: 看cardinality好像problem3特別高，但其實只是因為matching一定要一對一，所以n 40所有人都1對1就是20，但是problem 1, problem2 是看IDS，所以量級其實不太一樣。)

3. WS different p



- 反而是p比較有爭議，Requirement1-1, 1-2(aka problem1, problem2)都表現都差不多。
- GPT的解釋: 他認為0.7會特別噴一根很高的是因為他保有一定的ring的特性，但是又有點小混亂，導致在這個點反而半隨機半規則，進而導致她反而花了更多力氣才到好的解。
- 我認為或許也有一部分是因為我叫他n比較大的話trial就設定少一點，所以bias比較大這件事也有造成一定的影響。

---

5. AI Prompt (if used)

本作業在程式骨架規劃、各題 solver 設計、checker 撰寫與文件整理過程中，有使用生成式 AI 協助產生草稿；但對於數學定義、utility function、tie-breaking、exact checker 以及結果分析的方向，皆經過多次人工討論與修正後才納入實作。

以下附上本次作業中使用過的主要 prompts。

5.1 主程式骨架 prompt

```
請幫我產生一個「單一 Python 主程式」的骨架，用來完成一份 graph game 作業，但現在不要幫
我實作完整演算法，只要先把架構、介面、資料流、輸入輸出整理好，並保留清楚的 TODO。

需求與限制如下：

【整體目標】
我要一支 main.py，從 command line 讀入一張 graph，然後依序執行三個 graph games，最
後印出三個答案。
目前先不要實作完整求解邏輯，只要把架構搭好，讓我之後可以逐段補 solver。

【輸入格式】
程式要能從 command-line arguments 讀入 graph。
格式如下：
- 第一個參數是節點數 n
- 接著有 n 個長度為 n 的 bit string
- 例如：
  python main.py 6 010000 101100 010010 010010 001101 000010

請你實作：
1. parse_args()
2. validate_input_format()
3. build_graph_from_bitstrings()

【graph 表示方式】
請使用簡單、可讀性高的 Python 寫法：
- Graph 類別
- 內含：
  - n
  - adjacency_matrix
  - adjacency_list
  - degree(i)
  - neighbors(i)
  - is_edge(i, j)

節點在內部請統一使用 0-based index。
但如果註解要對應作業說明，可以補充人類閱讀時可視為 1-based node1, node2, ...

【主流程】
main() 應該：
1. 解析輸入
2. 建立 Graph
3. 依序呼叫三個 solver：
   - solve_mis_based_ids(graph)
   - solve_symmetric_mds_based_ids(graph)
   - solve_maximal_matching(graph)
4. 印出三個答案

【重要】
目前三個 solver 不要實作真正演算法。
請只做出完整函式介面、回傳資料格式、清楚註解、TODO 區塊。
```

```
你可以先讓每個 solver 回傳一個 dataclass，例如：
- cardinality: int
- move_count: int | None
- state: Any
- is_valid: bool

【程式風格】
1. 優先可讀性，不要 overengineering
2. 使用 Python standard library 即可
3. 加上型別註記
4. 加上足夠註解，說明每個區塊的用途
5. 對輸入錯誤要有清楚例外或錯誤訊息
6. 不要使用第三方套件
7. 不要偷放任何真正的 graph algorithm library
8. 先以作業架構為主，不要追求 fancy abstraction

【額外希望】
請加入：
- if __name__ == "__main__": main()
- 一個 pretty_print_results() 函式，統一輸出格式
- 註解中標出哪些地方之後要由我補 game logic

【輸出】
請直接輸出完整的 main.py 程式碼，不要省略。
```

5.2 WS generator prompt

```
請幫我產生一支 Python 程式 ws_generator.py，用來生成符合課堂作業需求的 Watts-
Strogatz (WS) model graph。

【目標】
這支程式只負責生成 graph 測資，不要跟三個 game solver 混在一起。

【功能需求】
請實作一個簡單清楚、可讀性高的 WS generator，包含：

1. generate_ws_graph(n, k, p, seed=None)
2. 輸出格式要能轉成作業要求的 bit-string graph format：
   - 第一個值是節點數 n
   - 接著是 n 個長度為 n 的 bit string
3. 提供 CLI 使用方式，例如：
   python ws_generator.py --n 20 --k 4 --p 0.2 --seed 42

【WS model 規則】
請嚴格按照這個版本實作：
1. 先建立一個 n-node regular ring graph
2. 每個 node 連到它左右最近的 k/2 個鄰居
3. 然後對原本 ring 上的每條邊，以機率 p 進行 rewiring
4. rewiring 時：
   - 保持 simple undirected graph
```

```
   - 不可 self-loop
   - 不可 duplicate edge
5. 最後必須保證：
   - 沒有 isolated node（每個點 degree >= 1）

【限制】
1. 假設 k 是偶數，且 0 < k < n
2. p 介於 [0, 1]
3. 使用 Python standard library
4. 不要使用 networkx 或任何第三方圖論套件
5. 程式重點是正確、可讀、容易檢查

【希望包含的函式】
- validate_parameters(n, k, p)
- build_ring_lattice(n, k)
- rewire_edges(...)
- ensure_no_isolated_nodes(...)
- adjacency_matrix_to_bitstrings(...)
- main()

【重要】
請在註解中說明：
- ring lattice 是怎麼建的
- rewiring 怎麼避免 duplicate/self-loop
- 為什麼要額外檢查 isolated node
- 若 generator 為了避免 isolated node 而進行修補，請寫得保守而清楚

【輸出】
請直接輸出完整 ws_generator.py 程式碼，不要省略。
```

5.3 Problem 1 solver prompt

```
請幫我在現有 Python 主程式架構中，補上 Problem 1: MIS-based IDS Game 的 solver。

請嚴格按照教材的以下定義實作，不要自行改寫規則：

1. state 是 bit vector C = (c_1, ..., c_n)，每個 c_i ∈ {0,1}
2. 對每個節點 i，定義
   L_i = { j | j 是 i 的鄰居，且 deg(j) >= deg(i) }
3. utility:
   u_i(C) = c_i * (1 - alpha * sum_{j in L_i} c_j)
   其中 alpha > 1
4. best response:
   - 若存在 j in L_i 且 c_j = 1，則 BR_i = 0
   - 否則 BR_i = 1

請你實作：
- 一個函式建出每個 i 的 L_i
- utility 計算函式
- best_response(i, state)
```

```
- 執行 best-response dynamics 直到收斂
- cardinality 計算
- validity check（至少檢查最後結果是否為 independent dominating set）
- multi-start execution：從多個初始 state 重跑，最後取 minimum cardinality 當作
Problem 1 輸出

實作要求：
- 使用 Python standard library
- 維持原本單一主程式結構
- tie-breaking 使用 random
- 請保留註解，清楚說明教材定義和程式的對應
- 不要使用第三方套件
- 不要實作其他兩題，這次只補 Problem 1
```

5.4 Problem 2 solver prompt

```
請幫我在現有的 Python 單檔主程式架構中，補上 Problem 2: Symmetric MDS-based IDS
Game 的 solver。這次只處理 Problem 2，請不要改動其他兩題的規則，也不要重構整個專案。

【背景】
這份作業是 graph game 作業。對於 Problem 2，我要實作的是教材中的 Symmetric MDS-
based IDS Game。請嚴格按照以下數學定義實作，不要自行發明新的 utility 或簡化成別的
heuristic。若有不確定之處，請保留清楚註解，不要自行腦補。

【狀態定義】
- graph 是無向圖
- 每個節點 i 對應一個玩家 p_i
- state C = (c_1, ..., c_n)
- 每個 c_i ∈ {0,1}
- c_i = 1 表示節點 i 被選進集合
- c_i = 0 表示節點 i 沒被選

【教材定義】
1. 定義 M_i = N_i ∪ {i}
2. 定義 v_i(C) = sum(c_j for j in M_i)
3. 給定常數 alpha > 1，定義
   g_i(C) = alpha if v_i(C) == 1 else 0
4. 給定常數 gamma > n * alpha，定義
   w_i(C) = sum(c_i * c_j * gamma for j in N_i)
5. 給定常數 beta，且 0 < beta < alpha
   玩家 i 的 utility 定義為：
   - 若 c_i == 1：
       u_i(C) = sum(g_j(C) for j in M_i) - beta - w_i(C)
   - 若 c_i == 0：
       u_i(C) = 0

【best response 規則】
Problem 2 不像 Problem 1 那樣有簡化後的 closed-form BR。
請直接使用以下方式實作 best response：
```

```
- 固定其他玩家狀態 c_-i
- 分別嘗試：
  1. 令 c_i = 0 時計算 utility
  2. 令 c_i = 1 時計算 utility
- 若 utility(1) > utility(0)，則 best response 為 1
- 若 utility(0) > utility(1)，則 best response 為 0
- 若兩者相等，請使用 random tie-breaking，在 0 和 1 中隨機選一個

【要求實作的函式】
請在現有架構中加入或補齊下列功能，命名可依原始程式微調，但功能要清楚：

1. build_closed_neighborhoods(graph)
2. domination_count(state, closed_neighbors, i)
3. domination_gain(state, closed_neighbors, i, alpha)
4. independence_penalty(graph, state, i, gamma)
5. utility_problem2(graph, state, i, alpha, beta, gamma, closed_neighbors)
6. best_response_problem2(graph, state, i, alpha, beta, gamma,
closed_neighbors, rng)
7. run_problem2_dynamics(...)
8. solve_symmetric_mds_based_ids(graph, ...)

【validity check】
請補一個 validity check，至少檢查 final state 是否為 independent dominating
set：
1. independent：
   - 任兩個相鄰節點不能同時都被選
2. dominating：
   - 每個節點不是自己被選，就是至少有一個鄰居被選

【multi-start 與 initial states】
請不要只跑一次。
請幫我實作一個簡單但清楚的 multi-start 策略，例如：
- all-zero initial state
- all-one initial state
- 多個 random bit vector initial states

【參數設定】
請在程式中用保守、清楚的方式設定預設值，例如：
- alpha = 2.0
- beta = 1.0
- gamma = n * alpha + 1.0
但請確保一定滿足：
- alpha > 1
- 0 < beta < alpha
- gamma > n * alpha
```

5.5 Problem 3 solver prompt

```
請幫我在現有的 Python 單檔主程式架構中，補上 Problem 3: Matching Game / Maximal
Matching 的 solver。請只處理 Problem 3，不要改動 Problem 1 與 Problem 2 的數學規
```

```
則，也不要大幅重構整份程式。

【作業背景】
這題的 graph 是無向圖。每個節點是一個 player。
每個 player i 的策略是：
- 選擇一個鄰居 j
- 或選擇 None（對應題目中的 null）

只有當：
- state[i] == j
- 且 state[j] == i
時，(i, j) 才形成一個 matched pair。

這題的目標是透過自定 utility function 與 best-response dynamics，讓最終狀態對應到 maximal matching。
我接受 heuristic / bias，但整體寫法必須仍然是「每個 player 比較策略 utility 後更新」，而不是直接改寫成純 greedy matching algorithm。

【state 表示】
請用：
- state[i] = None，表示玩家 i 選擇 null
- state[i] = j，表示玩家 i 選擇鄰居 j

【utility 設計】
請使用以下 utility 方案，不要自行改動成別的版本：

對於玩家 i，考慮策略 state[i] = s_i：

1. 若 s_i is None：
   utility = 0

2. 若 s_i = j 且 state[j] == i：
   表示成功 mutual match
   utility = 3 + bias(j)

3. 若 s_i = j 且 state[j] is None：
   表示我向一個目前 unmatched 的鄰居提案
   utility = 1 + bias(j)

4. 其他情況：
   例如我向一個不會回選我的鄰居提案，或對方已指向別人
   utility = -1 + bias(j)

其中 bias(j) 用來偏好 degree 較小的鄰居。
請使用簡單清楚的版本，例如：
   bias(j) = 1.0 / (1 + degree(j))

【best response】
對每個玩家 i，請枚舉所有可選策略：
- None
- graph.neighbors(i) 中的每個鄰居 j

固定其他玩家狀態不變，逐一計算 utility，
選 utility 最大的策略作為 best response。
```

```
若有 tie，請使用以下 tie-breaking：
1. 優先選 degree 較小的鄰居
2. 若仍 tie，使用 random
3. None 也要納入比較

【update order heuristic】
每輪 dynamics 中，先找出所有目前狀態不等於 best response 的 improvable players。
從這些玩家中選擇要更新的 player 時，請加入 heuristic：
1. 優先考慮目前尚未成功 matched 的玩家
2. 在這些玩家中，優先 degree 較小者
3. 若仍 tie，再 random

【需要實作的函式】
請加入或補齊以下函式（名稱可微調，但功能需清楚）：

1. is_mutually_matched(state, i)
2. matched_partner(state, i)
3. utility_problem3(graph, state, i)
4. best_response_problem3(graph, state, i, rng)
5. run_problem3_dynamics(graph, initial_state, rng, max_steps)
6. compute_matching_edges(state)
7. matching_cardinality(state)
8. is_valid_matching_state(graph, state)
9. is_maximal_matching_state(graph, state)
10. solve_maximal_matching(graph, ...)

【驗證】
請在 final result 中保留：
- final state
- move_count
- cardinality
- is_valid
- is_maximal
```

5.6 Checker / benchmark prompt

```
請幫我生成一支新的 Python 程式 checker.py。這支程式不是交作業用，而是我本機用來驗證與
分析 graph game solver 的工具。請不要修改我現有的 314581030_HW1_main.py 的
stdout 輸出格式，也不要把 checker 的邏輯硬塞回 main。

【目標】
checker.py 需要做到以下事情：
1. 自動呼叫 ws_generator.py 產生 WS graph 測資
2. 取得 graph 後，直接 import 並呼叫 314581030_HW1_main.py 中的 solver 函式，而
不是只看 stdout
3. 對三個問題分別做 exact checker
4. 蒐集多組 WS graph 的統計資料
5. 使用 matplotlib 繪製 performance 圖

【重要限制】
```

```
1. checker.py 是獨立程式
2. 不要修改 main.py 既有 stdout 格式
3. 可以 import main.py 中的：
   - Graph
   - build_graph_from_bitstrings
   - solve_mis_based_ids
   - solve_symmetric_mds_based_ids
   - solve_maximal_matching
4. 可以使用 Python standard library 與 matplotlib
5. 不要使用 networkx
6. 圖表請使用 matplotlib，不要使用 seaborn
7. 每張圖獨立繪製，不要做 subplot
8. 不要指定顏色，使用 matplotlib 預設即可

【與 ws_generator.py 的整合方式】
請優先用 subprocess 呼叫 ws_generator.py，例如：
python ws_generator.py --n 20 --k 4 --p 0.2 --seed 42

並解析其輸出成：
- n
- bitstrings

再利用 build_graph_from_bitstrings 建 Graph 物件。

【對 main solver 的整合方式】
取得 Graph 後，請直接呼叫：
- solve_mis_based_ids(graph)
- solve_symmetric_mds_based_ids(graph)
- solve_maximal_matching(graph)

不要只解析 main.py 的 stdout，因為 checker 需要 final state 才能做 exact
validation。

【Problem 1 / Problem 2 exact checker】
請實作共用 checker：
1. is_independent(graph, state)
2. is_dominating(graph, state)
3. is_independent_dominating_set(graph, state)

另外建議加上：
4. is_ne_problem1(...)
5. is_ne_problem2(...)

【Problem 3 exact checker】
請實作：
1. is_strategy_profile_well_formed(graph, state)
2. compute_matching_edges(state)
3. is_valid_matching(graph, state)
4. is_maximal_matching(graph, state)

【批次 benchmark】
請實作批次測試功能，例如：
1. benchmark_over_n(...)
2. benchmark_over_p(...)
```

```
每組參數可以跑多個 seeds，並統計：
- avg_cardinality_problem1
- avg_cardinality_problem2
- avg_cardinality_problem3
- avg_move_count_problem1
- avg_move_count_problem2
- avg_move_count_problem3
- valid_rate_problem1
- valid_rate_problem2
- valid_rate_problem3
- maximal_rate_problem3
```

Report Prompt

```
我希望你可以幫我把我們前面所有討論的內容整理出來，包含那幾個關鍵部件的prompt。變成一個符合作業的spec的格式(code description, all test cases you use (involving given one), result analysis, and AI prompt (if you use)) 然後我希望以繁體中文撰寫，你先寫成markdown格式然後我再自己增減內容轉換成正式版本
```

small_test_case

```
請先看我的src/下的所有檔案，包含314581030_HW1_main.py的程式碼以後，以正確的測資輸入分別產生以下內容，並且以--trials 30作為平均值產生輸出。
類型應包含
1. Path
2. Cycle
3. Star
4. Complete / dense graphs
5. 我們本來預設的範例測資。
```

討論紀錄(我透過與chatGPT討論，將前面的Prompt和Report完成)

https://chatgpt.com/share/69cbae71-129c-83a5-9934-166cf9175f8c

6. 結論

本作業嘗試以 graph game 的方式求解三個圖論問題。 前兩題主要透過教材既有的 utility function 與 best-response rule，形成合法的 IDS；第三題則需自行設計 utility function，並透過 matching legality、maximality 與 NE 等條件來驗證其合理性。

除了主程式外，我也另外實作了 WS generator 與 checker，以便對不同 graph 結構下的表現進行測試與分析。整體而言，本次作業不只是單純實作圖論演算法，而是要思考如何將圖論目標轉化為局部玩家的 utility 與策略更新規則，並藉由 checker 驗證最終穩定狀態是否真的對應到正確的圖論解。